

Answer Key — TaskFlow Senior Exam · Days 1–6

Instructor copy 100 MCQs Rationale + distractor traps

Position spread: A = 28 · B = 25 · C = 23 · D = 24 · no positional pattern, no run > 2.

Quick-reference grid

1.C	2.A	3.D	4.B	5.A	6.D	7.C	8.A	9.B	10.D
11.A	12.C	13.B	14.A	15.D	16.C	17.B	18.A	19.C	20.B
21.D	22.A	23.C	24.D	25.B	26.A	27.A	28.B	29.C	30.A
31.D	32.B	33.A	34.C	35.D	36.B	37.A	38.C	39.D	40.A
41.B	42.C	43.D	44.A	45.B	46.D	47.C	48.A	49.B	50.D
51.A	52.B	53.D	54.A	55.C	56.B	57.A	58.D	59.C	60.B
61.A	62.C	63.D	64.B	65.A	66.C	67.B	68.D	69.C	70.A
71.D	72.B	73.C	74.D	75.A	76.B	77.D	78.C	79.A	80.B
81.D	82.A	83.C	84.B	85.A	86.C	87.B	88.D	89.A	90.B
91.C	92.D	93.A	94.C	95.B	96.D	97.A	98.B	99.C	100.D

Day 1 — Clean Architecture

1. Answer: C — The dependency rule forbids the inner domain depending on an outer-layer library; coupling the contract to `Response/Dio` means the "abstraction" leaks the transport.

Traps: A names a real flaw (return type) but it's a symptom, not the root; B invents a language rule; D confuses import-path style with the dependency rule.

2. Answer: A — Use cases depend on the `TaskRepository` abstraction, not the transport; the GraphQL swap is confined to data.

Traps: B confuses entity with DTO; C thinks use cases see the data source; D is false — the mapper does change.

3. Answer: D — Dependency inversion opposes the two arrows: compile-time data→domain, runtime domain→data via the injected impl.

Traps: A says arrows align; B invokes reflection (it's constructor DI); C — domain must never import data, even in tests.

4. Answer: B — Two boundaries violated: presentation skips the use case/contract and exposes DTOs to the UI.

Traps: A invents a `StreamProvider` rule; C is a real Riverpod nuance but not the architectural defect; D reframes the DTO leak as harmless.

5. Answer: A — A contract in `data/` forces any domain code referencing it to import data — inward-pointing-outward.

Traps: B overstates (it compiles); C dismisses location as cosmetic; D fabricates a DI cycle.

6. Answer: D — Serialization is an API concern; baking `fromJson` into the entity makes the domain aware of the wire format.

Traps: A/C invent compile rules; B — the harm exists regardless of which core lib parses the date.

7. Answer: C — The mature answer weighs boilerplate cost vs change-resilience and ties the call to lifespan/team size.

Traps: A/B are dogmatic poles; D invents a cost asymmetry.

8. Answer: A — `d.toEntity()` enforces DTO→Entity; the `on ...Exception` clauses enforce Exception→Failure.

Traps: B (no validation/caching here); C reverses mapping direction; D — DI is constructor-based and it returns the Either.

9. Answer: B — `Either` makes failure a typed value the caller must handle; throwing relies on remembering to catch.

Traps: A is speculative perf; C misreads Either; D is false (exceptions cross async fine).

10. Answer: D — The lesson prescribes feature-first vertical slices.

Traps: A (a shared core doesn't fix the cross-feature tangle); B/C destroy layering.

11. Answer: A — Importing `package:flutter` couples the domain to the UI framework regardless of how "data-like" `Color` seems.

Traps: B — this `Color` is a framework import, not core; C invents a "types are exempt" loophole; D — `Equatable` is irrelevant.

12. Answer: C — Depend on the abstraction: a fake of the `TaskRepository` contract tests the use case in pure Dart.

Traps: A/D drag Dio into the domain; B just relocates the test.

13. Answer: B — The DTO mirrors the wire format and the mapper translates, so a rename is absorbed in those two files.

Traps: A is the anti-pattern; C — use cases never see JSON keys; D — contracts express operations, not field names.

14. Answer: A — `fpdart` exports its own `Task`; `hide Task` removes that symbol so the name resolves unambiguously to the entity.

Traps: B reverses what `hide` does; C — it won't compile without it; D invents an `Either` collision.

15. Answer: D — Without boundaries the data concern is smeared across widgets, multiplying change-sites.

Traps: A blames state mgmt; B fixing the token leaves JSON scattered; C — JSON lib is irrelevant to a transport change.

16. Answer: C — Contract in `domain/` (business promise), impl in `data/`, inverting the dependency.

Traps: A is the #1 beginner error; B — the UI doesn't own data contracts; D drags data concerns into domain.

17. Answer: B — Use cases are the single orchestration point per business action; skipping them scatters cross-cutting logic.

Traps: A calls them ceremonial (disproved by the pain); C trivializes it as allocation cost; D — the dependency rule isn't the harm.

Day 2 — Networking & API Client

18. Answer: A — Request-side `onRequest` runs in registration order, so logging at index 0 captures the request before auth adds the token.

Traps: B invents reverse response-order; C — logging adds no token; D — request interceptors run forward.

19. Answer: C — A `receiveTimeout` can occur after the server committed the write, so retrying a non-idempotent POST can double-create.

Traps: A — POST isn't idempotent; B — a timeout doesn't guarantee non-receipt; D — reads don't corrupt a cache.

20. Answer: B — 403 = authenticated but unauthorized; a fresh token grants no new permission, so refresh-and-retry loops pointlessly.

Traps: A expired = 401; C validation = 422; D wrong URL = 404.

21. Answer: D — The refresh call's own 401 re-enters `onError`, recursing forever (no guard flag).

Traps: A — no duplicate-URL short-circuit; B — `resolve` only fires on success; C — no try/catch here, awaiting doesn't bound recursion.

22. Answer: A — Concurrent uncoordinated refreshes race and can rotate tokens out from under each other; refresh once and queue the rest.

Traps: B assumes harmless; C — status codes aren't client-downgraded; D — logging out on a recoverable 401 destroys the UX.

23. Answer: C — `connectTimeout` bounds connection establishment; `receiveTimeout` bounds waiting for response data.

Traps: A swaps with `sendTimeout`; B — not the whole round trip / not DNS-only; D — they're distinct phases.

24. Answer: D — A fresh `Dio()` has an empty interceptor list, so requests silently skip auth/retry/error translation.

Traps: A/B — multiple Dios on one baseUrl is legal; C — interceptors are per-instance, not inherited.

25. Answer: B — `SharedPreferences` is plaintext, readable on compromised devices/backups; tokens belong in secure storage.

Traps: A — not wiped on upgrade; C — failure is confidentiality, not latency; D — it supports many keys.

26. Answer: A — A connect-phase timeout on an idempotent GET is transient with no server effect — safe and likely to help.

Traps: B 422 will reject the same body; C 404 is definitive; D 400 is malformed and will fail again.

27. Answer: A — Translating at the border keeps `dio` out of inner layers.

Traps: B fabricated await rule; C letting `DioException` reach the UI is the leak; D Dart has no checked exceptions.

28. Answer: B — Logging after auth records the populated `Authorization` header — a real token leak in release.

Traps: A this order is correct for sending; C logging transmits nothing; D logging doesn't decode JWTs.

29. Answer: C — Without `handler.next` the chain stalls and the request `Future` never settles.

Traps: A no implicit continuation; B it doesn't proceed at all; D Dio doesn't throw, it waits.

30. Answer: A — Dio's default `validateStatus` accepts only 2xx, so a 500 travels the `onError` path.
Traps: B a 500 is non-2xx; C only one path runs; D the response isn't dropped.

31. Answer: D — A retry-recovered request must resolve as success; error-mapping must not turn it into a typed error.
Traps: A/B couple retry and the data source's catch incorrectly; C a recovered request should surface zero errors.

32. Answer: B — PUT (full replace) is idempotent; POST (create) duplicates on replay.
Traps: A PUT does reach the DB; C inverts the truth; D POST isn't idempotent and wire speed is irrelevant.

33. Answer: A — Dio is constructor-injected, so a mock/adaptor returning 401 exercises the real catch-and-translate.
Traps: B it's injected, not internal; C Dio is mockable; D overriding skips the logic under test.

34. Answer: C — With `sendTimeout` unset, a stalled request-body upload isn't bounded by connect/receive timeouts and can hang.
Traps: A connect already succeeded; B receive hasn't started; D Dio doesn't substitute `receiveTimeout` for send.

Day 3 — DTOs, Serialization & Mapping

35. Answer: D — The DTO mirrors the wire format; the `String?` field stays the raw string until the mapper parses it.
Traps: A thinks the DTO parses dates; C thinks a matching key needs `@JsonKey`; B confuses `@Default` with a nullable requirement.

36. Answer: B — A renamed JSON key is absorbed by the DTO's `@JsonKey`; Dart field name, entity, mapper, and UI are unchanged.
Traps: A mapper reads the Dart field name; C is the architecture-less anti-pattern; D leaks API naming into the entity.

37. Answer: A — Keep the DTO literal: `DateTime.parse` won't handle `"28/06/2026"`, and `"1" → bool` interpretation belongs in the mapper.
Traps: B eliminating the mapper recouples to the API; C the syntax is valid; D misses the real defect.

38. Answer: C — `freezed` gives the data-class machinery; `json_serializable` produces `.g.dart` (de)serialization, triggered by declaring `fromJson`.
Traps: A the named misconception; B/D nonsensical.

39. Answer: D — `--delete-conflicting-outputs` overwrites stale generated files. *Traps: A never edit generated files; C breaks `_$...FromJson`; D[...] — abandoning codegen isn't a fix.* (the wrong "D-ish" options are A/B/C here.)

40. Answer: A — A missing key for a `required` non-nullable field makes `fromJson` throw; model it `String?`.
Traps: B `required` has no default; C the crash is at parse time; D `build_runner` generates fine.

41. Answer: B — Parsing (DTO) vs interpretation (mapper) are distinct concerns; a literal DTO isolates format/API change.
Traps: A `DateTime` is a legal `freezed` field; C perf is irrelevant; D `DateTime.parse` is global.

42. Answer: C — Two literal versioned DTOs + two mappers → one stable entity; domain/UI never learn of the versions.

Traps: A leaks versioning into the UI; B pollutes the entity; D nullability can't reconcile a renamed/retyped key.

43. Answer: D — `DateTime.TryParse` returns `null` on bad input (unlike `parse` which throws) → graceful degradation.

Traps: A they differ; C `tryParse` returns `DateTime?`; B no epoch fallback.

44. Answer: A — Skipping the mapper lets the API-shaped DTO reach the UI, so wire churn ripples into widgets.

Traps: B DTOs must not leak regardless of `@JsonKey`; C watch mode is unrelated; D `@Default` vs `required` is orthogonal.

45. Answer: B — Nested objects/list elements deserialize by delegating to each type's own `fromJson`.

Traps: A it isn't automatic; C ignoring drops the data; D `json_serializable` builds typed lists.

46. Answer: D — DTOs stay at the boundary; only entities circulate internally.

Traps: A violates the rule; C the mapper translates, it doesn't mint DTOs; D[reverse] — entities never carry DTOs (that's option B).

47. Answer: C — Generated `fromJson` looks for a key literally named `title`; with only `todo` present the required field is absent and parsing fails.

Traps: A no fuzzy matching; B never stores the key name as value; D `build_runner` doesn't infer renames.

48. Answer: A — The crash is in `fromJson`: the DTO claimed non-null for a field the API sends as null; declare `String?`.

Traps: B try/catch hides not fixes; C regeneration won't change a declared type; D entity can't force the API.

49. Answer: B — `fromJson` parses (untyped→typed DTO); the mapper translates (DTO→entity).

Traps: A swaps the roles; C they're different concerns; D `fromJson` yields a DTO, not the entity.

50. Answer: D — `toIso8601String()` is the canonical wire format that round-trips through `tryParse`; `toString()` uses a non-interchange format and dropping `?.` crashes on null.

Traps: A they differ; B no epoch mandate; C reverses which is wire-safe.

51. Answer: A — Generated files are recreated on every build; durable logic belongs in the DTO declaration or mapper.

Traps: B passing tests don't make a doomed edit durable; C gitignore doesn't stop overwrite; D `watch` regenerates too.

Day 4 — Repository & Data Sources

52. Answer: B — Returning `List<TaskDto>` leaks the data layer's shape into the domain; return entities.

Traps: A caching ordering is fine; C converting is the repo's job; D the cache legitimately stores DTOs.

53. Answer: D — Caching/branching policy in the UI means each screen applies slightly different rules → contradictions.

Traps: A a flush bug wouldn't split by screen; C mapping is deterministic; D[cursors] would change which tasks appear, not flip one's status.

54. Answer: A — SSoT = one coordination point (the repository) owning the policy, not one data source.

Traps: B/C miscount sources; D inverts the layering.

55. Answer: C — A data source must do one narrow job; coordinating remote+local and returning `Either` is the repository's role.

Traps: A a minor robustness nit; B the cache stores DTOs by design; D error granularity is a refinement.

56. Answer: B — Option B hits network → caches → maps → falls back to cache, returning `NetworkFailure` only when cache is empty.

Traps: A(opt A) no fallback → blank screen; C(opt C) cache-first serves stale while online; D(opt D) returns DTOs and `Right([])` on empty, hiding failure.

57. Answer: A — The remote source maps the `connectionError` to `NetworkException`; the repo catches it and falls back.

Traps: B nothing raw escapes; C connection ≠ server failure; D a thrown exception means no parsed body.

58. Answer: D — The repository is the data/domain seam; converting there keeps the domain exception-free while sources stay dumb.

Traps: A `Failure` is importable in data; B sources use try/catch routinely; C use cases should never see exception types.

59. Answer: C — The contract is "sources throw, repository converts"; returning `Either` smears error policy across layers.

Traps: A it compiles; B uniform `Either` violates the split; D `Failure` left makes it worse.

60. Answer: B — Stub remote to throw, stub local to return, assert mapped `Right`, and `verify` the cache was read.

Traps: A real Hive/Dio = integration test; C stubs the wrong path; D leaving remote unstubbed never triggers the branch.

61. Answer: A — Returning the unchanged cached entity on a failed write silently fakes success.

Traps: B catching is fine, the return is the flaw; C a toggle returns `Task`, not `Unit`; D cursor is irrelevant.

62. Answer: C — One tested rule in the repository = correctness-by-construction + testability; no two screens can disagree.

Traps: A a perf footgun but not the core; B false — the UI *could* misuse data types; D hot reload is unrelated.

63. Answer: D — The blank-then-fill symptom means successes aren't cached, so there's no last-known data to show instantly.

Traps: A a success-returns-failure bug would show an error; B mapping a list is trivial CPU; C cache-first would feel fast, not blank.

64. Answer: B — Mapping belongs in the repository (the seam), keeping DTOs out of the domain.

Traps: A a source returning entities pulls domain into data; C use cases consume entities; D mapping in the widget is the worst leak.

65. Answer: A — Cache on success + fall back to cache only in the `NetworkException` branch; leave the 401/Server mappings untouched.

Traps: B/D cache-first inverts the spec; C falling back on a 401 masks an auth failure.

66. Answer: C — Wiring local→remote makes a data source depend on the other; coordination belongs in the repository.

Traps: A latency is secondary; B it's not architecturally fine; D nothing technically prevents it — it's a design objection.

67. Answer: B — A god-repository does the HTTP/DB/parsing itself instead of delegating to narrow sources.

Traps: A returning Either is correct; C coordinating two sources is its purpose; D per-method mapping is expected.

68. Answer: D — Stubbing a method doesn't prove it was called; without `verify(...).called(1)` the test passes even with no caching, and it never checks the entity.

Traps: A a stub permits but doesn't require a call; B/C miss the real gap.

Day 5 — Caching, Offline-First & Pagination

69. Answer: C — Cache-first serves instantly and only hits the network on a miss — ideal for rarely-changing, staleness-tolerant data.

Traps: A confuses with must-be-current; B write-through is a write strategy; D SWR wastes a call per read here.

70. Answer: A — `difference > ttl` is true when the entry is *stale*, so the code serves cache exactly when expired — inverted.

Traps: B UTC cancels in a difference; C an equivalent form, not the bug; D fetching first defeats the cache.

71. Answer: D — An in-memory queue dies on force-close; it must be persisted to replay after restart.

Traps: A optimistic UI is correct; B no conflict occurred; C that's network-first.

72. Answer: B — Offset paging + top inserts shift every row, so a page boundary re-serves a seen row; a cursor would be stable.

Traps: A missing `hasMore` → empty requests at the end; C rebuild replaces, not duplicates; D TTL triggers refetch, not per-row dupes.

73. Answer: C — `_items = page.items` discards prior pages, and no `isLoadingMore` guard lets a fast scroll fire overlapping calls.

Traps: A/B/D invert best practices (cursors, off-UI-thread parsing).

74. Answer: D — A must-be-correct balance calls for network-first with cache as fallback.

Traps: A SWR briefly shows stale money; B cache-first can show outdated indefinitely; C write-through is for writes.

75. Answer: A — SWR serves cache instantly and revalidates in the background; first paint never blocks.

Traps: B blocking is the misconception SWR avoids; C it depends on serving the cache; D that's write-then-read.

76. Answer: B — Only field merge or asking the user preserves both edits — at real implementation cost.

Traps: A LWW discards the loser; C server-wins discards the offline edit; D explicit data loss.

77. Answer: D — Need a `hasMore` flag; once a page is empty / `nextCursor` is null, stop firing requests.
Traps: A/D[offset] don't fix it; B the guard is present; C a bigger page only delays the end.

78. Answer: C — Write locally + update UI immediately, enqueue a *persisted* action, flush when online.
Traps: A skipping persistence loses the change; B that's network-first; D blocking is anti-offline-first.

79. Answer: A — A cursor points at a specific item, so "after this one" survives inserts/deletes; an offset is a shifting position.
Traps: B absolute index = the offset that drifts; C cursors don't cache the set; D they tolerate change, not freeze it.

80. Answer: B — Refresh should reset to page 1 and fetch only that page (cached), not refetch every loaded page.
Traps: A still wastes 6 round trips; C that's load-more; D reproduces the freeze.

81. Answer: D — Cursor paging needs `nextCursor` (and an end signal) returned so the caller can advance and stop.
Traps: A reintroduces offsets; B can't advance; C total count is optional.

82. Answer: A — No TTL → the cached list is always "fresh" and never revalidated, so external deletions never propagate.
Traps: B SWR does refresh; C cursors are unrelated; D network-first would over-fetch, not show stale forever.

83. Answer: C — Local-primary + deferred sync = *eventual* consistency, which is why conflict resolution exists.
Traps: A that's strong consistency; B correctness is very much a concern; D contradicts local-as-authoritative.

84. Answer: B — The optimistic local entry and the server copy are two records; reconcile/replace by ID after sync.
Traps: A offset would be worse; C TTL governs freshness, not identity; D missing `hasMore` causes empty requests.

Day 6 — Error Handling & Authentication

85. Answer: A — Above the repository everything speaks `Failure`; a raw exception escaping would be an untyped surprise no one is forced to handle.
Traps: B invents an isolate rule; C use cases *can* catch; D downplays the leak as convention.

86. Answer: C — An unwrapped `DioException` matches no `on` clause and escapes the repository — the exact leak `Either` prevents (fix: trailing `catch (_)`).
Traps: A no swallow path; B Dart doesn't auto-route to the last clause; D `await` doesn't transmute types.

87. Answer: B — `Left` = failure, `Right` = success; the callbacks are swapped, so failures get returned raw and successes go to `showError`.
Traps: A order is failure-first; C `fold` takes two callbacks; D inverts the convention.

88. Answer: D — Tunnel → `NetworkException` → `Left(NetworkFailure)` → UI folds to "No internet connection."
Traps: A is Tarek's untranslated bug; B is the generic-message trap; C `fold` runs offline fine.

89. Answer: A — The frequently-exposed access token is short-lived; the precious refresh token is long-lived and minting-only — a security trade-off.

Traps: B it's security, not load balancing; C reverses the transmission pattern; D the model is HTTP-client-independent.

90. Answer: B — The refresh token is the long-lived, high-value credential; leaking it from plaintext lets an attacker mint access tokens for its whole lifetime.

Traps: A SharedPreferences isn't encrypted; C it stores strings fine; D location matters despite expiry.

91. Answer: C — The `/refresh` 401 re-enters `onError` → another refresh → loops forever (no guard).

Traps: A `fetch` doesn't ignore 401; B Dio has no auto-recursion breaker; D the loop is network-bound, not an instant stack overflow.

92. Answer: D — Detect an in-flight refresh, queue the other nine, refresh once, then replay all ten.

Traps: A ten parallel refreshes is the storm; B dropping nine requests is a failure; D[independent] assumes server dedup the client can't rely on.

93. Answer: A — Expired refresh token = genuine session end → hard logout: clear tokens, reset state, redirect to login (remember destination).

Traps: B surfaces a raw exception; C infinite background retry never recovers; D leaving a stale token is insecure.

94. Answer: C — The `User` entity models identity; tokens are transient credentials for secure storage, not coupled to who the user is.

Traps: A Equatable handles strings; B the interceptor reads tokens from storage; D session state isn't tracked by diffing a token.

95. Answer: B — Never store passwords; the refresh token exists precisely to be a revocable, minting-only long-lived secret.

Traps: A encryption doesn't justify it; C base64 is encoding, not protection; D a plaintext backup is worse.

96. Answer: D — `Either` makes failure a value the type system forces you to handle; throwing relies on remembering to catch.

Traps: A perf isn't the point; B throwing is legal under null safety; C `Either` represents success as `Right`.

97. Answer: A — Silent refresh needs an `onError` that catches 401, refreshes, and retries; this interceptor only injects the token.

Traps: B injection alone isn't refresh; C every request carries the access token correctly; D no retry logic exists to loop.

98. Answer: B — A guard flag set but never cleared makes the interceptor believe a refresh is always in flight, so every later 401 is suppressed forever.

Traps: A/C/D describe unrelated failures (leak, unattached token, password storage).

99. Answer: C — Two transformations: the data source *throws* an exception, the repository *catches and returns* it as `Left(Failure)`; above that it stays a `Failure` value.

Traps: A/B/D misplace where the type changes.

100. Answer: D — Collapsing every exception into one `ServerFailure` discards actionable distinctions (offline banner, re-login, retry) — the generic-message trap.

Traps: A it's not cleaner; B re-throwing to the UI is the original leak; C `ValidationFailure` isn't a universal target.

How to grade yourself

- **90–100:** senior-ready on the data layer + auth. Interview-strong.
- **75–89:** solid; revisit the 2–3 weakest days.
- **< 75:** re-read those days' lessons and retake — focus on the *why*, not the keyword.

Highest-value questions to review if missed

#3, #8 (dependency inversion / the two boundaries), #21, #91, #92 (refresh loop + storm — the senior differentiators), #37, #43, #50 (DTO-literal discipline), #56, #65 (read-through control flow), #71, #84 (persisted queue + optimistic reconciliation).